

MoE Router 正交损失的等价变换与子图融合

脱敏分享版 | 本文讨论通用的 MoE Router 辅助损失、数学等价变换与 `torch.compile` 子图融合方法，不包含内部模型、代码路径或运行信息。

结论先行

一个训练热路径未必需要手写 CUDA kernel 才能优化。某 MoE Router orth-loss 的 Gram GEMM 本身已经高效，真正的开销来自每层反复构造 identity，以及 normalization、margin、pointwise、reduction 等碎算子。

通过恒等式

$$||G - I||^2 = ||G||^2 - 2 \cdot \text{trace}(G) + E$$

可以消除 identity；margin 路径则计算全矩阵 excess，再减去 diagonal excess，保持只累计非对角项。随后用 `torch.compile(fullgraph=True)` 融合周边子图，保留 Gram GEMM。最终 GPU operation 从 17 降至 7，隔离的目标函数 forward+backward 提升 18.77%，四卡 MoE Layer proxy 的 forward+backward 提升 3.97%，value/gradient 9/9 通过。

目标函数与 full-layer proxy 使用的 shape 和测量范围不同，两组绝对时间不能直接横比。这里的 3.97% 只代表四卡 MoE Layer proxy，不是完整训练 step 收益；融合开关保持默认关闭，首次编译成本也不计入 steady-state 数据。

1. Orth-loss 在做什么

设 Router 权重或其归一化表示为矩阵 W ，构造 Gram 矩阵：

$$G = \text{normalize}(W) \cdot \text{normalize}(W)^T$$

理想情况下，不同 expert 的 Router 方向应尽量解耦。一个常见目标是让 G 接近单位阵 I ：

$$L = ||G - I||^2$$

实际实现还可能引入 margin，只惩罚超过阈值的非对角相关性。这个损失通常不是整网最大算子，但会在每个 MoE 层重复执行；小开销乘上几十层后就值得处理。

2. Trace 告诉我们的不是“GEMM 太慢”

对 steady-state forward 做分解后，可以看到：

```
normalize
-> Gram GEMM
-> create identity
-> subtract / abs / margin
-> diagonal mask
-> square / sum / scale
```

单次 forward 共 17 个 GPU operation，其中 Gram GEMM 只有 2 个。也就是说，热点不是“大矩阵乘不够快”，而是周边的 identity、pointwise、reduction 与 launch 碎片。

这类问题若直接替换 GEMM，往往收益有限；更合适的顺序是先找数学冗余，再决定编译融合边界。

3. Margin=0：直接消除单位阵

利用 Frobenius 范数展开：

$$\begin{aligned} & ||G - I||^2 \\ &= \text{trace}((G - I)^T(G - I)) \\ &= ||G||^2 - 2 \cdot \text{trace}(G) + ||I||^2 \\ &= ||G||^2 - 2 \cdot \text{trace}(G) + E \end{aligned}$$

其中 E 是 expert 数， $||I||^2 = E$ 。这样无需显式创建 $E \times E$ identity，也无需完整执行 $G-I$ 。

伪代码如下：

```
gram = normalized @ normalized.T
loss = gram.square().sum() - 2 * gram.diagonal().sum() + num_experts
```

公式只是等价变换，仍需检查 dtype、归约顺序和 scale 是否与旧实现一致。

4. Margin 路径：全矩阵计算后减去对角项

当损失只统计超过 margin 的非对角相关性时，最直观实现会创建 mask 或 identity。更轻量的方法是：

```
all_excess = penalty(G, margin).sum()
diag_excess = penalty(diag(G), margin).sum()
loss = all_excess - diag_excess
```

这样仍然只累计 off-diagonal 项，但避免构造完整 diagonal mask。需要特别验证 margin 的符号、绝对值定义、阈值边界与梯度，因为这些细节决定公式是否真正等价。

5. 为什么经历了三个版本

5.1 V1: in-place diagonal

第一版直接原地修改对角元素，单函数 forward 看起来更快。但 profile 中同时出现了 `CopySlices` backward，提示 autograd 路径的额外工作可能抵消前向收益；四卡 MoE Layer proxy 的 forward+backward A/B 实测下降 3.16%。由于本轮没有单独消融 `CopySlices` 的成本，不将完整回退唯一归因于它。

这是典型的“前向 microbenchmark 胜利、训练真实路径失败”。因此 V1 明确判定 NO-GO。

5.2 V2: identity-free reduction

第二版使用等价式消除 identity，语义正确，完整层接近持平。它证明数学路线可行，但 launch 与 pointwise 仍然分散，收益不足以承担新路径维护成本。

5.3 V3: identity-free + compiled subgraph

第三版在 V2 基础上使用 `torch.compile(fullgraph=True)`，融合 normalization 与 Gram GEMM 前后的 pointwise/reduction，Gram GEMM 本身继续由成熟后端执行。最终版本不是一个“大而全”的手写 kernel，而是“等价式缩图 + 编译器融合周边 + 保留高效 GEMM”。

6. 性能与正确性

6.1 隔离目标函数与四卡完整层 proxy

前两行是隔离 orth-loss 目标函数的计时；第三行是另一组 shape 与测量范围下的四卡 MoE Layer proxy。它们分别回答“子图本身是否变快”和“收益能否传递到完整层代理”，不应用绝对时间做跨表归因。

指标	Legacy	Compiled	变化
Target forward	0.117588 ms	0.082559 ms	+29.79%
Target forward+backward	0.478209 ms	0.388468 ms	+18.77%
Four-GPU MoE Layer proxy forward+backward	17.674392 ms	16.973511 ms	+3.97%
Target training peak memory	90.005 MiB	80.003 MiB	-11.11%

6.2 结构变化

指标	Legacy	Compiled
GPU operations / steady forward	17	7
Gram GEMM	2	2
周边执行	多个 Torch kernel + memset	5 个融合 kernel

Profiler 中的 projected duration 可用于比较结构变化，但不与 CUDA Event latency 横向混用。最终 GO 指标应来自无 profiler 的多轮事件计时和完整层验证。

6.3 正确性矩阵

测试覆盖不同 dtype、规模和 margin：

- value 与 gradient：8/8；
- compiled 目标 shape forward+backward：1/1；
- 合计：9/9 PASS；
- 实际 Router dispatch 与四卡完整层 proxy smoke：PASS。

7. Compile 路径的工程约束

7.1 首次编译成本

首次启用会有秒级 compile 开销；steady-state benchmark 必须明确 warmup，并单独报告冷启动。对于短任务或 shape 高频变化的场景，编译成本可能抵消运行收益。

7.2 Shape cache

相同 Router shape 可复用 compiled graph；dynamic shape 是否产生重编译、cache 膨胀或 graph break，必须独立验证。

7.3 默认关闭

新路径通过 feature flag opt-in，默认仍走 eager。这使尚未覆盖的 dynamic shape、CUDA Graph 与训练配置拥有明确回退路径。

8. 这次优化为什么有效

收益来自三个因素的组合：

1. **数学缩图**：不再创建和消费 identity；

2. **Launch 收敛**: 周边碎算子由 17 个 operation 收敛到 7 个;

3. **保留成熟 GEMM**: 没有为追求“全融合”而重写已高效的矩阵乘。

如果只做其中任意一项, 完整层收益都可能不足。V1/V2 的负结果正好说明, 优化必须从 autograd 和完整层视角验收; 但未做独立对照的 profile 线索, 不应被写成唯一根因。

9. 不能外推的结论

- 3.97% 是另一 shape/scope 下的四卡 MoE Layer forward+backward proxy, 不是 full training step。
- Target 函数的 18.77% 与 full-layer proxy 的 3.97% 只能在各自 matched A/B 内解释, 不是同一对象的两级直接分解。
- Steady-state 不包含首次 compile。
- 当前结果不证明 dynamic shape、所有 margin 或 CUDA Graph 均已覆盖。
- GPU operation 下降不等于等比例 wall-time 下降。

10. 可复用检查表

- ☐ 目标损失能否通过迹、范数或对角项重写, 消除临时张量?
- ☐ 前向优化是否在 backward 引入 CopySlices、额外保存或 graph break?
- ☐ 编译边界是否保留成熟 GEMM, 只融合周边碎算子?
- ☐ 是否分别测 target、完整 layer 与完整 step?
- ☐ 是否报告首次 compile 和 steady-state?
- ☐ value、gradient、dtype、margin 与真实 dispatch 是否都覆盖?
- ☐ 是否提供默认关闭和 eager fallback?

11. 总结

这类训练热路径的关键不是“把所有东西写进一个 CUDA kernel”, 而是准确识别计算图中真正冗余的部分。先用数学等价式消除 identity, 再让编译器融合适合融合的 pointwise/reduction, 同时保留成熟 GEMM, 可以用较低维护成本获得可传递到完整层的收益。负版本、编译成本和未覆盖边界也必须与正收益一起记录。